Problem 1

(a)(i)

$$T(0) = \Theta(1), \qquad T(n) = T(n-1) + \Theta(\log n) \quad \text{for } n \geq 1.$$

(a)(ii)

$T(n) \in \Theta(n \log n)$. Backward substitution gives $T(n) = T(0) + \sum_{i=1}^{n} \Theta(\log i) = \Theta\big(\log_2(n!)\big)$, using $\sum_{i=1}^{n} \log_2 i = \log_2 \prod_{i=1}^{n} i = \log_2(n!)$.

By Stirling's formula, $n! = \sqrt{2\pi n}\,(n/e)^n\big(1 + \Theta(1/n)\big)$, so $\log_2(n!) = n\log_2 n - n\log_2 e + O(\log n) = n\log_2 n - \Theta(n)$. The subtracted term is positive, so $\log_2(n!) \leq n\log_2 n$ and $\log_2(n!) \in O(n\log n)$; and since $\Theta(n) = o(n\log n)$, we get $\log_2(n!) \geq n\log_2 n - o(n\log_2 n)$, so $\log_2(n!) \in \Omega(n\log n)$. Hence $\log_2(n!) \in \Theta(n\log n)$ and therefore $T(n) \in \Theta(n\log n)$.

(b)(i)

return $2p + q,\ 2q$

(b)(ii)

$T(k) \in \Theta(k)$. Each invocation of *TotalLightCountBitsRec* does constant work and makes one recursive call on $k-1$, so $T(k) = T(k-1) + \Theta(1)$ with $T(1) = \Theta(1)$, which unwinds to $\Theta(k)$. *TotalLightCountBits* adds one call and constant work, so it is $\Theta(k)$ as well.

(b)(iii)

TotalLightCountBits is more efficient, by a factor of $\Theta(n)$: $\Theta(\log n)$ against $\Theta(n\log n)$.

Let $n = 2^k$, so $k = \log_2 n$. The integers representable with k bits are exactly $0, \ldots, 2^k - 1 = 0, \ldots, n-1$, i.e. all the tram numbers except n itself; and $n = 2^k$ has exactly one light on. So the answer to the problem of part (a) is

$$\mathit{TotalLightCountBits}(\log_2 n) + 1.$$

Its cost is $\Theta(\log n)$ to obtain the binary representation of n, plus $\Theta(k) = \Theta(\log n)$ for the recursion by (b)(ii): in total $\Theta(\log n)$.

Problem 2

(a)(i)

$$\mathit{arr} = [1,\ 1] \qquad \text{and} \qquad \mathit{start} = 0.$$

Since no entry of *arr* is 0, the network has no terminal stop, so the configuration is invalid and the procedure must return **False**. *IsValid_AI* instead recurses forever, so it never returns **False**.

(a)(ii)

The disjunction **or** is evaluated from left to right, so the call *IsValid_AI*(arr, $i + dist$) is made first.

1. *IsValid_AI*([1, 1], 0): $i = 0$; $0 \le i < 2$; $arr[0] = 1 \neq 0$; $dist \leftarrow 1$; evaluate the first disjunct, i.e. call ...
2. *IsValid_AI*([1, 1], 1): $i = 1$; $0 \le i < 2$; $arr[1] = 1 \neq 0$; $dist \leftarrow 1$; evaluate the first disjunct, i.e. call ...
3. *IsValid_AI*([1, 1], 2): $i = 2 \ge length(arr) = 2$, so **return False**.
4. Back in call 2: the first disjunct is **False**, so evaluate the second one, i.e. call *IsValid_AI*([1, 1], 0) — and we are back in call 1.
5. *IsValid_AI*([1, 1], 0): identical to call 1; evaluate the first disjunct ...
6. *IsValid_AI*([1, 1], 1): identical to call 2; evaluate the first disjunct ...
7. *IsValid_AI*([1, 1], 2): **return False**.
8. Back in call 6: evaluate the second disjunct, i.e. call *IsValid_AI*([1, 1], 0) — and the whole pattern starts again.

Calls 5–8 repeat calls 1–4 with exactly the same arguments, so the sequence of calls $0, 1, 2, 0, 1, 2, \ldots$ continues forever and the top-level call never returns.

(b)

A depth-first search from *start*, marking the stops already visited.

Algorithm 1

```
 1: function IsValid(arr, start)
 2:     n ← length(arr)
 3:     if start < 0 or start ≥ n then
 4:         return False
 5:     visited ← new Boolean array with indices 0 ... n − 1
 6:     for i ← 0 to n − 1 do
 7:         visited[i] ← False
 8:     return Search(arr, start, visited)

 9: function Search(arr, i, visited)              ▷ visited is shared by all calls
10:     if i < 0 or i ≥ length(arr) then
11:         return False
12:     if visited[i] then
13:         return False
14:     visited[i] ← True
15:     if arr[i] = 0 then
16:         return True
17:     dist ← arr[i]
18:     if Search(arr, i + dist, visited) then
19:         return True
20:     return Search(arr, i − dist, visited)
```

(c)

Since *visited*[i] is set before the recursive calls and never reset, at most n calls to *Search* get past the visited test. Each of those does $O(1)$ work and makes at most two more calls, so in total at most $2n + 1$ calls are made, each taking $O(1)$. Adding the $\Theta(n)$ initialisation of *visited* gives $O(n)$.

Problem 3

(a)

Within t minutes tram i makes $\lfloor t/time[i] \rfloor$ trips.

Algorithm 2

```
1: function MaxTrips(time, t)
2:     n ← length(time)
3:     trips ← 0
4:     for i ← 0 to n − 1 do
5:         trips ← trips + ⌊t/time[i]⌋
6:     return trips
```

MaxTrips$(time, \cdot)$ is *non-decreasing* in t, since every $\lfloor t/time[i] \rfloor$ is.

(b)

Algorithm 3

```
1: function MinTimeBruteForce(time, m)
2:     t ← 1
3:     while MaxTrips(time, t) < m do
4:         t ← t + 1
5:     return t
```

The loop terminates: writing $t_{\min} = \min_i time[i]$, the fastest tram on its own makes m trips within $t_{\min} \cdot m$ minutes, so *MaxTrips*$(time, t_{\min} m) \geq m$ and the loop stops at some $t \leq t_{\min} \cdot m$.

(c)

Let T be the value returned. The loop condition is evaluated once for each of $t = 1, 2, \ldots, T$, and each evaluation runs *MaxTrips*, which costs $\Theta(n)$; the rest of each iteration is $O(1)$. So the running time is $\Theta(n \cdot T)$.

Since $time[i] \leq k$ for every i,

$$\begin{aligned} MaxTrips(time, k\lceil m/n \rceil) &= \sum_i \left\lfloor \frac{k\lceil m/n \rceil}{time[i]} \right\rfloor \\ &\geq\ n\lceil m/n \rceil\ \geq\ m, \end{aligned}$$

so $T \leq k\lceil m/n \rceil$; and this bound is attained when every tram is as slow as possible, $time[i] = k$ for all i, for then *MaxTrips*$(time, t) = n\lfloor t/k \rfloor$, which first reaches m at $t = k\lceil m/n \rceil$. Hence the worst-case running time is

$$\Theta(n \cdot k\lceil m/n \rceil) = \Theta(k(m+n)), \qquad \text{since } m \leq n\lceil m/n \rceil < m + n.$$

(d)

By (a) the values *MaxTrips*$(time, 1)$, *MaxTrips*$(time, 2), \ldots$ form a sorted array, and T is the position of the first entry that reaches m. Binary search finds that position. Part (b) supplies the right end of the range, $t_{\min} \cdot m$.

Algorithm 4

```
1:  function MinTimeBinarySearch(time, m)
2:      n ← length(time)
3:      t_min ← time[0]
4:      for i ← 1 to n − 1 do
5:          if time[i] < t_min then
6:              t_min ← time[i]
7:      lo ← 1
8:      hi ← t_min · m                          ▷ MaxTrips(time, hi) ≥ m, so T ∈ [lo, hi]
9:      while lo < hi do
10:         mid ← ⌊(lo + hi)/2⌋
11:         if MaxTrips(time, mid) ≥ m then
12:             hi ← mid
13:         else
14:             lo ← mid + 1
15:     return lo
```

(e)

Since computing $t_{\min}$ takes $\Theta(n)$, and the search interval initially contains $hi - lo + 1 = t_{\min} \cdot m \leq k\,m$ candidate times (recall $t_{\min} \leq k$), and every iteration at least halves it, there are at most $\lceil \log_2(km) \rceil + 1$ iterations. Each iteration is dominated by its single call to *MaxTrips*, i.e. $\Theta(n)$; the rest is $O(1)$. Hence the worst-case running time is

$$\Theta(n) + O\big(n \log(km)\big) = O\big(n \log(km)\big) = O\big(n(\log k + \log m)\big).$$